

SPDATA - Serviço de Processamento de Dados LTDA



Capacitação de SQL (Criação de Query)

23 de maio de 2017

Objetivo: Capacitação dos profissionais para gestão e melhorias nos processos de utilização do sistema SGH®

Romilton Gonçalves Dias
Gerente de Capacitação

Banco de dados Relacional

Bancos de dados são sistemas destinados a armazenar grandes volumes de dados, provendo um acesso rápido e fácil aos dados. Existem diversas categorias de bancos de dados, das quais a mais importante e conhecida são os bancos de dados relacionais. Estes se caracterizam basicamente por organizar as informações em tabelas (onde cada linha da tabela é um registro), estabelecendo ligações entre os dados destas tabelas. Dos conceitos de banco de dados existentes, três são importantes para entender como funciona e também compreender a razão de ocorrerem determinados problemas.

Estes conceitos são: chave primária (PK) chave estrangeira (FK) e índices. Outros serão apresentados

Primary Key (chave Primária)

Toda tabela em um banco de dados relacional deve conter uma chave primária (Primary Key). Esta tem a função de identificar de forma única cada linha de uma tabela. Ou seja, pela chave primária pode-se saber exatamente de qual linha da tabela está-se falando Exemplo: Um documento do Estoque não pode haver o mesmo ano, mês e doc. Obs: Pode ser localizado na opção Unique Constraints caso for Único

Foreign Key (chave estrangeira)

Uma chave estrangeira (Foreign Key) estabelece uma relação entre duas tabelas.

Um exemplo de chave estrangeira que temos em nosso sistema é o relacionamento entre as tabelas de lançamentos do SADT (SILANEXA) e a tabela de atendimentos (SICADATE) Só será possível realizar um lançamento se tiver o atendimento

Índice

Um índice em um banco de dados tem duas funções básicas:

Melhorar a performance do acesso aos dados, provendo uma maneira rápida de encontrar a informação desejada e também garantir a integridade, ou seja, a coerência dos dados no banco de dados.

Para garantir que os conceitos de chave primária e chave estrangeira sejam respeitados, o banco de dado estabelece índices para cada chave estrangeira e chave primária criados.

ID (O que é o ID e para que serve?)

O ID tem a função de identificar um registro de maneira única e rápida já que todos os IDs são indexados geralmente por padrão o ID é uma PK e único.

Simplifica o relacionamento entre tabelas, evitando a repetição de chaves compostas em diferentes tabelas.

Serve para criar uma dependência com outras tabelas.

Facilidade em localizar um arquivo

Serve para aumentar a integridade das informações

TIPOS DE DATOS

BIGINT	Integer, 64 bits (-2×10^{63} hasta $2 \times 10^{63} - 1$)
CHAR(<i>n</i>)	Cadena de N caracteres.
DATE	Entero de 32 bits. 01-01-100 to 31-12-9999
DECIMAL (<i>precision</i> [, <i>scale</i>])	Decimal (Precisión: 1-18, escala: 1-18) DECIMAL(8,3)=ppppp.sss
DOUBLE PRECISION	Punto Flotante de 64 bits 2.225×10^{-308} hasta 1.797×10^{308}
FLOAT	Punto Flotante de 32 bits 1.175×10^{-38} hasta 3.402×10^{38}
INTEGER	Entero de 32 bits, con signo -2.147.483.648 hasta 2.147.483.647
NUMERIC (<i>precision</i> [, <i>scale</i>])	Igual que DECIMAL (Precision [,scale])
SMALLINT	Entero de 16 bits (-32.768 hasta 32.767)
TIME	Entero de 32 bits 0:00:00 hasta 23:59:59.9999
TIMESTAMP	Entero 64 bits.
VARCHAR(<i>n</i>)	Cadena de n caracteres (0 to 32.765 bytes). Charset character size determines the maximum number of character that can fit in 32k.
BLOB	Variable. Dynamically sizable data type for storing large data such as graphics, text, and digitized voice. Blob. Blob subtype describes Blob contents.

SQL (Structured Query Language, ou Linguagem de Consulta Estruturada), é a linguagem de pesquisa declarativa padrão para banco de dados relacional.

A linguagem SQL é dividida em subconjuntos de acordo com as operações que queremos efetuar sobre um banco de dados.

- [DDL - Linguagem de Definição de Dados](#)
- [DML - Linguagem de Manipulação de Dados](#)

DDL - Linguagem de Definição de Dados

Linguagem de definição de dados (ou **DDL**, de *Data Definition Language*) é um conjunto de comandos dentro da SQL usada para a definição das estruturas de dados, fornecendo as instruções que permitem a criação, modificação e remoção das tabelas, assim como criação de índices. Estas instruções SQL permitem definir a estrutura de uma base de dados, incluindo as linhas, colunas, tabelas, índices, e outros metadados.

Entre os principais comandos DDL estão CREATE (Criar), DROP (deletar) e ALTER (alterar).

Exemplos: CREATE DATABASE meu_banco_de_dados

DML - Linguagem de Manipulação de Dados

Linguagem de manipulação de dados (ou **DML**, de *Data Manipulation Language*) é o grupo de comandos dentro da linguagem SQL utilizado para a recuperação, inclusão, remoção e modificação de informações em bancos de dados.

Os principais comandos DML são SELECT (Seleção de Dados), INSERT (Inserção de Dados), UPDATE (Atualização de Dados) e DELETE (Exclusão de Dados).

Resumo

A linguagem SQL está dividida em subconjuntos de acordo com as operações que queremos efetuar sobre um banco de dados.

- Linguagem de definição de dados (ou DDL, de *Data Definition Language*) é um conjunto de comandos dentro da SQL usada para a definição das estruturas de dados. Entre os principais comandos DDL estão CREATE (Criar), DROP (deletar) e ALTER (alterar).
- Linguagem de manipulação de dados (ou DML, de *Data Manipulation Language*) é o grupo de comandos dentro da linguagem SQL utilizado para a recuperação, inclusão, remoção e modificação de informações em bancos de dados. Os principais comandos DML são SELECT (Seleção de Dados), INSERT (Inserção de Dados), UPDATE (Atualização de Dados) e DELETE (Exclusão de Dados).

Cláusulas

As cláusulas são condições de modificação utilizadas para definir os dados que deseja selecionar ou modificar em uma consulta.

- **FROM** – Utilizada para especificar a tabela que se vai selecionar os registros.
- **WHERE** – Utilizada para especificar as condições que devem reunir os registros que serão selecionados.
- **GROUP BY** – Utilizada para separar os registros selecionados em grupos específicos.
- **HAVING** – Utilizada para expressar a condição que deve satisfazer cada grupo.
- **ORDER BY** – Utilizada para ordenar os registros selecionados com uma ordem específica.
- **DISTINCT** – Utilizada para selecionar dados sem repetição.
- **UNION** - combina os resultados de duas consultas SQL em uma única tabela para todas as linhas correspondentes.

Operadores Lógicos

- **AND** – E lógico. Avalia as condições e devolve um valor verdadeiro caso ambos sejam corretos.
- **OR** – OU lógico. Avalia as condições e devolve um valor verdadeiro se algum for correto.
- **NOT** – Negação lógica. Devolve o valor contrário da expressão.

Operadores relacionais

O SQL possui operadores relacionais, que são usados para realizar comparações entre valores, em estruturas de controle.

<	Menor
>	Maior
<=	Menor ou Igual
>=	Maior ou Igual
=	Igual
<>	Diferente

- **BETWEEN** – Utilizado para especificar valores dentro de um [intervalo fechado](#).
- **LIKE** – Utilizado na comparação de um modelo e para especificar registros de um banco de dados. "Like" + extensão % significa buscar todos resultados com o mesmo início da extensão.
- **IN** - Utilizado para verificar se o valor procurado está dentro de um« »a lista. Ex.: valor IN (1,2,3,4).

Funções de Agregação

As funções de agregação, como os exemplos abaixo, são usadas dentro de uma cláusula SELECT em grupos de registros para devolver um único valor que se aplica a um grupo de registros.\

AVG – Utilizada para calcular a média dos valores de um campo determinado.

COUNT – Utilizada para devolver o número de registros da seleção.

SUM – Utilizada para devolver a soma de todos os valores de um campo determinado.

MAX – Utilizada para devolver o valor mais alto de um campo especificado.

MIN – Utilizada para devolver o valor mais baixo de um campo especificado.

MONTH – Retorna o mês corrente

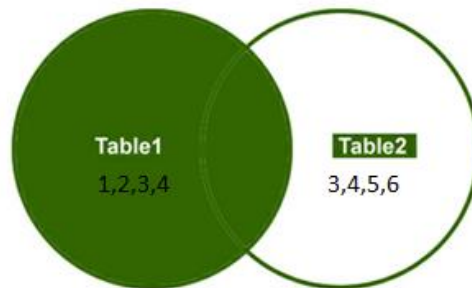
Year - Retorna o ano corrente

Junções

Tipos de Joins: LEFT JOIN, RIGHT JOIN, INNER JOIN

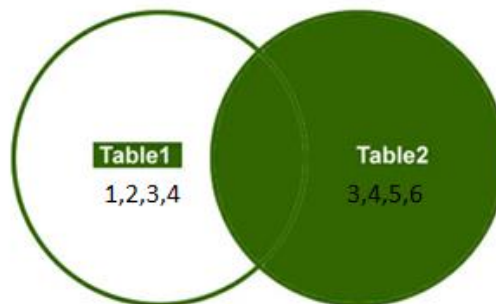
left join:

Como podemos observar, e a própria sintaxe indica, essa cláusula trabalha com os dados da tabela "Esquerda" como sendo os dados principais, ou seja, de acordo com o exemplo abaixo, o LEFT JOIN mostrará o que está na geitens (esquerda), podendo trabalhar também com qualquer outro dado da gevalmem com a mesma chave encontrada na geitens



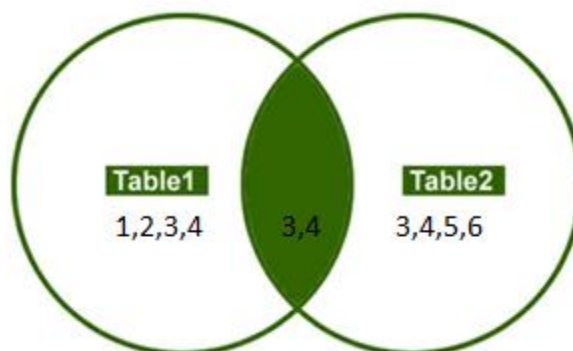
right join

retorna o que estiver na Tabela1 e Tabela2 com a mesma chave, e sendo o inverso do LEFT JOIN a tabela principal se torna a tabela da " Direita ", gevalmem

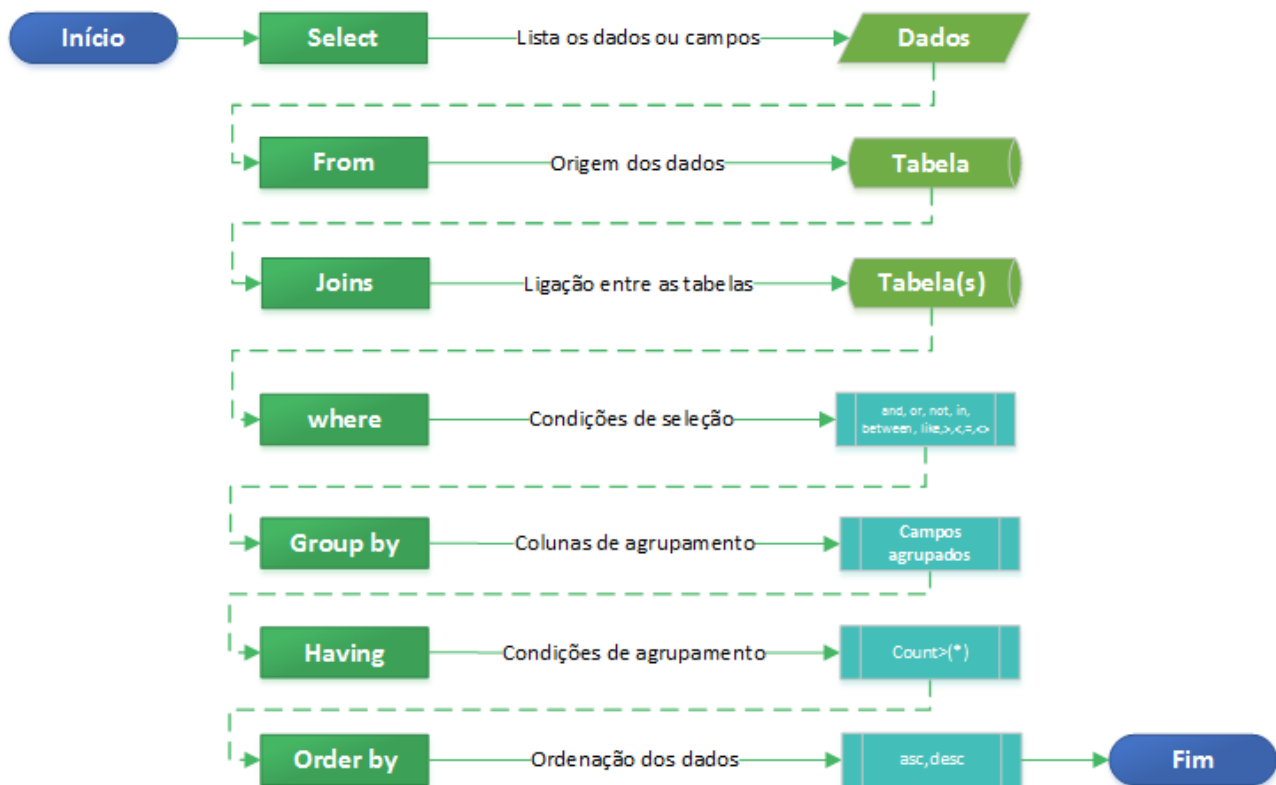


inner join

Retorna apenas o que está na geitens e gevalmem com a mesma chave.



Estrutura de Query (select)



Tabelas de envolvidas

Tabela	Módulo	Função da Tabela
ATCABECATEND	Atendimento	Tabela de cabeçalho de atendimento
RICADPAC	Atendimento	Tabela de cadastro de pacientes
TBLEITO	Tabelas	Tabela de cadastro de leitos
TBCONVEN	Tabelas	Tabela de cadastro de convênios
TBPROCTO	Tabelas	Tabela de procedimentos médicos
TBCID10	Tabelas	Tabela de CID
TBESPEC	Tabelas	Tabela de especialidades
TBCBOPRO	Tabelas	Tabela de CBO dos profissionais
TBCENCUS	Tabelas	Tabela de CDCs (centro de custos)
TBPROCED	Tabelas	Tabela de procedência no atendimento
TBCLINICA	Tabelas	Tabela de clínicas
TBUNIDAD	Tabelas	Tabela de unidades
TBMOTALT	Tabelas	Tabela de motivo de altas
GEITENS	Estoque	Tabela de Itens do estoque
GECADNFS	Estoque	Tabela do cabeçalho das entradas do estoque
GELANNFS	Estoque	Tabela de lançamento dos itens da nota fiscal na entrada
TBFORNEC	Tabelas	Tabela de fornecedores
GECADSAI	Estoque	Tabela de cadastro de documentos de saída do estoque
GELANSAL	Estoque	Tabela de lançamento dos itens de saída do estoque
GENONCOM	Estoque	Tabela de nomes comerciais dos itens do estoque
FCLANEXT	Faturamento	Tabela de lançamentos dos itens na conta do paciente
FCCTAEXT	Faturamento	Tabela de cadastro da conta no faturamento de convênios

Exemplos de comandos SQL

SELECT * FROM RICADPAC

(Lista todos os registros da tabela de pacientes)

SELECT NOME, MAE, NASC FROM RICADPAC

(Lista somente nome, mãe e data de nascimento da tabela de pacientes)

SELECT NOME, MAE, NASC FROM RICADPAC WHERE NASC >='01.01.2016'

(Lista somente quem nasceu de 01/01/2016 em diante)

SELECT NOME, MAE, NASC FROM RICADPAC WHERE NASC >='01.01.2016' AND NOME LIKE 'MARIA %'

(Lista somente quem nasceu de 01/01/2016 em diante e que o nome inicie com Maria)

SELECT COUNT(*) FROM RICADPAC WHERE NASC >='01.01.2016'

(conta todos os registros da tabela dos pacientes nascidos de 01/01/2016 em diante)

SELECT DISTINCT(CIDADE) FROM RICADPAC

(lista todas as cidades do cadastro de pacientes trazendo apenas uma de cada)

Exemplos práticos do dia a dia

SELECT COUNT(*) FROM ATCABECATEND WHERE DATA_HORA_ENTRADA=CURRENT_DATE

(conta a quantidade de pacientes atendidos no dia)

Sub select

Utilizado nos campos a alistar ou na condição where

SELECT PRONT, NOME, MAE, NASC FROM RICADPAC WHERE NASC >='01.01.2016' AND NOT EXISTS (SELECT * FROM ATCABECATEND WHERE RICADPAC.ID=ID_RICADPAC)

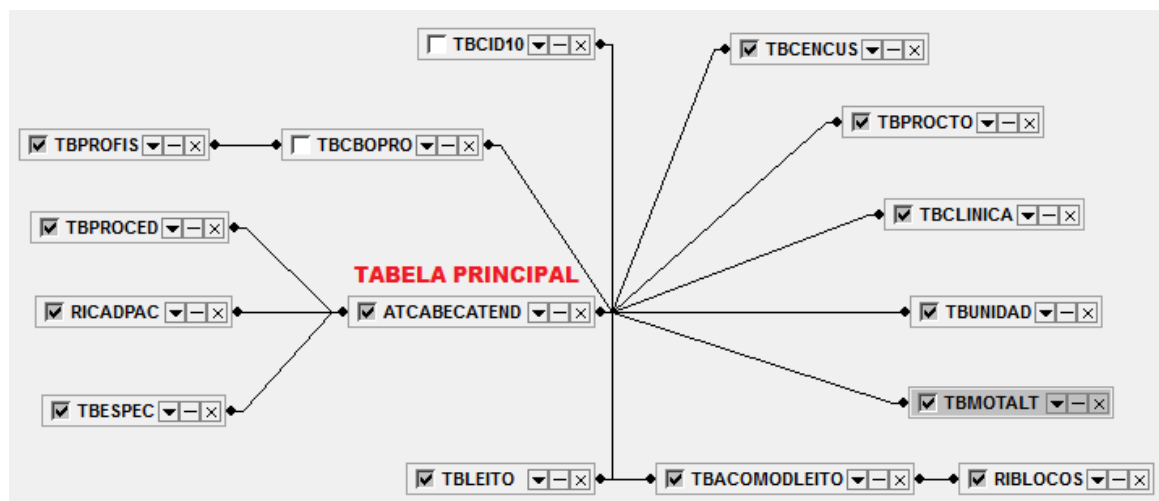
(lista somente quem nasceu de 01/01/2016 em diante e que não tenha um registro gerado na tabela de atendimentos) com subconsulta

***SELECT DISTINCT (SELECT NOME FROM TBCONVEN WHERE
TBCONVEN.COD=ATCABECATEND.ID_TBCONVEN)
FROM ATCABECATEND***

WHERE ATCABECATEND.DATA_HORA_ENTRADA BETWEEN '01.01.2016' AND '10.01.2016'

(Lista os convênios que tiveram atendimentos no período mostrando sem repetir o nome)

PROCESSO DE ATENDIMENTO TABELA PRINCIPAL ATCABECATEND



CAMPOS DE LIGAÇÃO

TABELAS AUXILIARES

JUNÇÕES

ID

ID_RICADPAC	=	RICADPAC	ID
ID_TBLEITO	=	TBLEITO	ID
ID_TBCONVEN	=	TBCONVEN	COD
ID_TBPROCTO	=	TBPROCTO	ID
ID_TBCID10	=	TBCID10	ID
ID_TBESPEC	=	TBESPEC	COD
ID_TBCBOPRO_ATENDIMENTO	=	TBCBOPRO	ID
ID_TBCENCUS	=	TBCENCUS	COD
ID_TBPROCED	=	TBPROCED	COD
ID_TBCLINICA	=	TBCLINICA	COD
ID_TBUNIDAD	=	TBUNIDAD	COD
ID_TBMOTALT	=	TBMOTALT	COD
ID_TBLEITO	=	TBLEITO	ID

Exemplo do sql acima

```
Select ricadpac.nome, ricadpac.nasc, atcabecatend.cod_atendimento,
atcabecatend.data_hora_entrada, tbprofis.nome, tbproced.nome, tbcencus.cod, tbcencus.nome,
tbespec.nome, tbleito.cod_leito, tbprocto.cod_procedimento, tbprocto.nome, tbclinica.nome,
tbunidad.nome, tbunidad.cod, tbmotalt.cod, tbmotalt.nome, tbacomodleito.cod_acomodacao,
riblocos.bloco, riblocos.nome
```

```
from atcabecatend
```

```
inner join ricadpac on (atcabecatend.id_ricadpac = ricadpac.id)
```

```
inner join tbespec on (atcabecatend.id_tbespec = tbespec.cod)
```

```
inner join tbproced on (atcabecatend.id_tbproced = tbproced.cod)
```

```
inner join tbcbopro on (atcabecatend.id_tbcbopro_atendimento = tbcbopro.id)
```

```
inner join tbprofis on (tbcbopro.id_tbprofis = tbprofis.id)
```

```
inner join tbcencus on (atcabecatend.id_tbcencus = tbcencus.cod)
```

```
inner join tbcid10 on (atcabecatend.id_tbcid10_principal = tbcid10.id)
```

```
inner join tbleito on (atcabecatend.id_tbleito = tbleito.id)
```

```
inner join tbacomodleito on (tbleito.id_tbacomodleito = tbacomodleito.id)
```

```
inner join riblocos on (tbacomodleito.id_riblocos = riblocos.bloco)
```

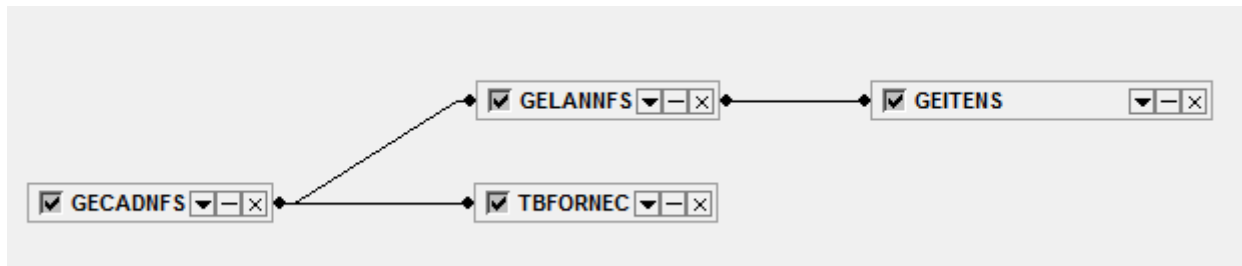
```
inner join tbprocto on (atcabecatend.id_tbprocto = tbprocto.id)
```

```
inner join tbclinica on (atcabecatend.id_tbclinica = tbclinica.cod)
```

```
inner join tbunidad on (atcabecatend.id_tbunidad = tbunidad.cod)
```

```
inner join tbmotalt on (atcabecatend.id_tbmotalt = tbmotalt.cod)
```

ENTRADA DE MERCADORIAS



CAMPOS DE LIGAÇÃO

TABELAS AUXILIARES

JUNÇÕES

GECADNFS

ANO	=	GELANNFS	ANO
MES	=	GELANNFS	MÊS
DOC	=	GELANNFS	DOC
FORN	=	TBFORNEC	COD

GELANNFS

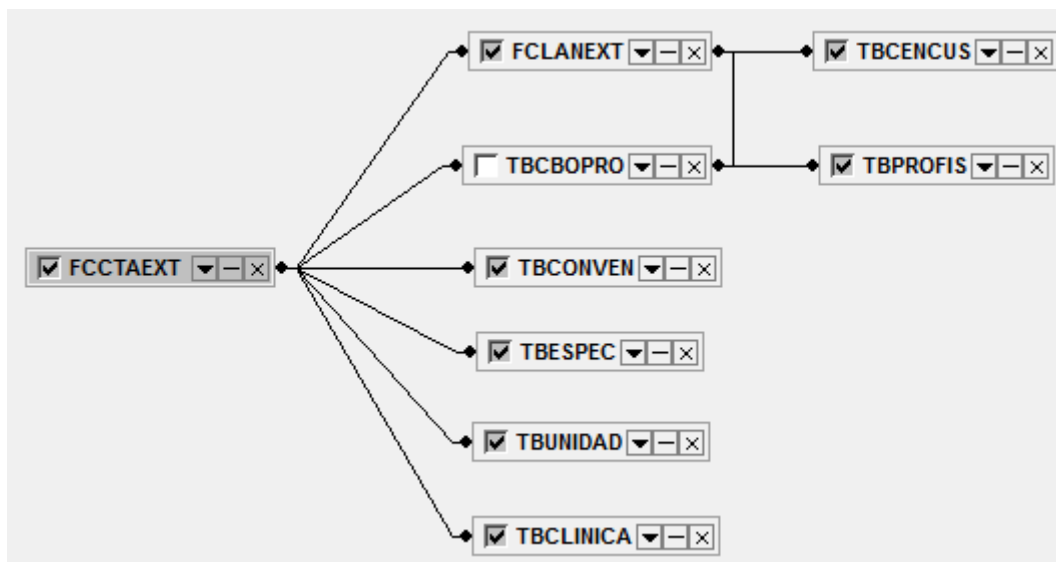
ITEM	=	GEITENS	COD
-------------	---	---------	------------

Sql do exemplo acima

```

select  tbfornece.nome,gecadnfs.nf,gelannfs.item,geitens.nome, geitens.cod,geitens.classif
from    gecadnfs
inner  join gelannfs on (gecadnfs.ano = gelannfs.ano) and (gecadnfs.mes = gelannfs.mes) and
(gecadnfs.doc = gelannfs.doc)
inner  join geitens on (gelannfs.item = geitens.cod)
inner  join tbfornece on (gecadnfs.forn = tbfornece.cod)
  
```

Faturamento de Convênios



CAMPOS DE LIGAÇÃO
FCCTAEXT

TABELAS AUXILIARES

JUNÇÕES

ID	=	FCLANEXT	ID_FCCTAEXT
CON	=	TBCONVEN	COD
CDC	=	TBCENCUS	COD
ESPEC	=	TBESPEC	COD
CLINICA	=	TBCLINICA	COD
UNIDAD	=	TBUNIDAD	COD
ID_TBCBOPRO_PRINCIPAL_CONTA	=	TBCBOPRO	ID
CID10	=	TBCID10	ID
ID_TBPROCTO	=	TBPROCTO	ID

Exemplo do sql acima

select

fcctaext.anopro,fcctaext.mespro,fcctaext.cod,fcctaext.paciente,fcctaext.con,fcctaext.cencus,fcctaext.esp,fcctaext.clinica,fcctaext.unid,fcctaext.usu_emi,fclanext.procto,fcctaext.alt,tbconven.nome,tbcencus.nome,tbespec.nome,tbclinica.nome,tbunidad.nome, fcctaext.tipo_cta,fclanext.tipo,tbprofis.nome

from tbunidad

inner join fcctaext on (tbunidad.cod = fcctaext.unid)

inner join fclanext on (fcctaext.id = fclanext.id_fcctaext)

inner join tbcencus on (fclanext.cdc = tbcencus.cod)

inner join tbcbopro on (fclanext.id_tbcbopro_solicitante = tbcbopro.id)

inner join tbprofis on (tbcbopro.id_tbprofis = tbprofis.id)

inner join tbconven on (fcctaext.con = tbconven.cod)

inner join tbespec on (fcctaext.esp = tbespec.cod)

inner join tbclinica on (fcctaext.clinica = tbclinica.cod)

where

((fcctaext.anopro = 2016)

and

(fcctaext.mespro = 3))